Lab Work: Dictionary in Python

1. Online Shopping Order Analytics

Problem Statement

An e-commerce company stores product sales data as:

```
sales = {
  "Laptop": 15,
  "Mouse": 45,
  "Keyboard": 32,
  "Monitor": 12,
  "Headphones": 28,
  "Printer": 8,
  "Webcam": 20,
  "Speaker": 18,
  "Tablet": 10,
  "Router": 25
}
```

Tasks

1. Display products sold more than 20 times.
2. Find the best-selling product.
3. Find the least-selling product.
4. Calculate total products sold.
5. Create a list of products requiring promotion (sales < 15).
6. Count products having sales between 10 and 30.

Sample Output

```
Products Sold More Than 20 Times:
Mouse
Keyboard
Headphones
Router

Best Selling Product: Mouse (45)

Least Selling Product: Printer (8)

Total Units Sold: 213

Products Requiring Promotion:
['Monitor', 'Printer', 'Tablet']

Products Having Sales Between 10 and 30: 6
```

---

2. Employee Performance Dashboard

Problem Statement

Employee performance scores are stored as:

```
performance = {
  "EMP101": 92,
  "EMP102": 78,
  "EMP103": 45,
  "EMP104": 88,
  "EMP105": 97,
  "EMP106": 56,
  "EMP107": 81,
  "EMP108": 64,
  "EMP109": 39,
  "EMP110": 73
}
```

Tasks

1. Display employees scoring above 80.
2. Count employees needing improvement (score < 60).
3. Find the top performer.
4. Calculate average performance score.
5. Create separate lists:
    - Excellent (≥ 90)
    - Good (75–89)
    - Average (60–74)
    - Poor (< 60)

Sample Output

```
Employees Scoring Above 80:
EMP101
EMP104
EMP105
EMP107

Top Performer: EMP105 (97)

Employees Needing Improvement: 3

Average Score: 71.3

Excellent:
['EMP101', 'EMP105']

Good:
['EMP102', 'EMP104', 'EMP107']

Average:
['EMP108', 'EMP110']
```

Poor:
['EMP103', 'EMP106', 'EMP109']

---

3. City Temperature Monitoring System

Problem Statement

Daily temperatures of different cities are stored as:

```
temperature = {
  "Delhi": 41,
  "Mumbai": 33,
  "Chennai": 37,
  "Kolkata": 39,
  "Bengaluru": 28,
  "Pune": 30,
  "Jaipur": 42,
  "Lucknow": 40,
  "Hyderabad": 35,
  "Ahmedabad": 43
}
```

Tasks

1. Display cities having temperature above 40°C.
2. Find the hottest city.
3. Find the coolest city.
4. Calculate average temperature.
5. Create a list of pleasant cities (temperature < 35°C).
6. Count cities with temperature between 35°C and 40°C.

Sample Output

Cities Above 40°C:
Delhi
Jaipur
Ahmedabad

Hottest City: Ahmedabad (43°C)

Coolest City: Bengaluru (28°C)

Average Temperature: 36.8°C

Pleasant Cities:
['Mumbai', 'Bengaluru', 'Pune']

Cities Between 35°C and 40°C: 4

---

4. Cricket Tournament Statistics

Problem Statement

Runs scored by players in a tournament:

```
runs = {
  "Virat": 645,
  "Rohit": 512,
  "Gill": 698,
  "Rahul": 435,
  "Hardik": 278,
  "Pant": 534,
  "Surya": 389,
  "Jadeja": 301,
  "Iyer": 455,
  "KL": 410
}
```

Tasks

1. Display players scoring more than 500 runs.
2. Find the Orange Cap winner.
3. Find the lowest scorer.
4. Calculate total runs scored.
5. Create a list of players scoring below 400.
6. Count players scoring between 400 and 600 runs.

Sample Output

```
Players Scoring More Than 500 Runs:
Virat
Rohit
Gill
Pant

Orange Cap Winner: Gill (698)

Lowest Scorer: Hardik (278)

Total Tournament Runs: 4657

Players Scoring Below 400:
['Hardik', 'Surya', 'Jadeja']

Players Between 400 and 600 Runs: 5
```

---

5. Smart Electricity Billing System

Problem Statement

Monthly electricity consumption (units) is stored as:

```
units = {
  "House101": 320,
```

```
  "House102": 180,
  "House103": 510,
  "House104": 275,
  "House105": 150,
  "House106": 430,
  "House107": 220,
  "House108": 390,
  "House109": 145,
  "House110": 600
}
```

Tasks

1. Display houses consuming more than 400 units.
2. Find the highest-consuming house.
3. Find the lowest-consuming house.
4. Calculate total units consumed.
5. Create lists:
    - Low Consumption (< 200)
    - Medium Consumption (200–400)
    - High Consumption (> 400)
6. Count houses eligible for an energy-saving campaign (consumption > 300).

Sample Output

```
Houses Consuming More Than 400 Units:
House103
House106
House110

Highest Consumption:
House110 (600 units)

Lowest Consumption:
House109 (145 units)

Total Units Consumed: 3220

Low Consumption:
['House102', 'House105', 'House109']

Medium Consumption:
['House101', 'House104', 'House107', 'House108']

High Consumption:
['House103', 'House106', 'House110']

Eligible for Energy-Saving Campaign: 5
```